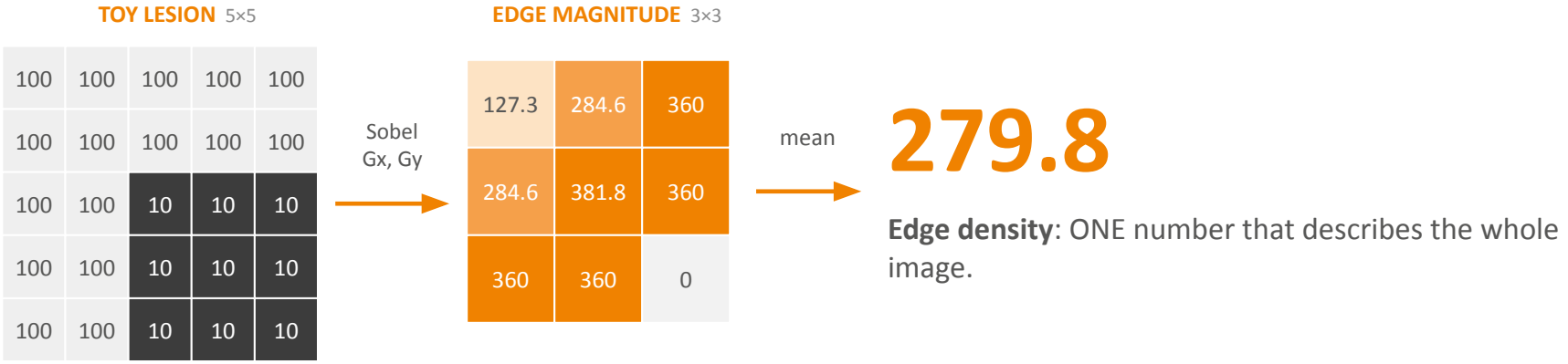


eada business school barcelona

Classical Machine Learning for Vision

Where Session 4 Left Us



- Session 4 ended with a question: does the mean edge strength differ between benign and malignant lesions?
- That single number is a **feature**: a measurement that summarizes an image.
- Today we compute many such numbers per image, stack them into a vector, and let a **classical ML model** learn the decision.

SESSION 5

Learning Objectives

1. **Explain** why a classical baseline normally comes before any deep learning model.
2. **Extract** color, histogram and texture features from an image: by hand, then in NumPy.
3. **Assemble** one feature vector per image into a matrix X and a label vector y.
4. **Train & compare** majority-class, logistic regression, kNN and random forest with scikit-learn.
5. **Evaluate honestly** confusion matrix, per-class recall, macro-F1, and the test set used only once.

SESSION 5

Part 1 — Theory

From pixels to feature vectors to predictions

eada business school barcelona

Why a Baseline Before Deep Learning?

? *Before reaching for a CNN, what's the cheapest possible thing you could try first?*

1 Is it learnable?

Does a simple signal (color, edges) predict the diagnosis at all?

2 Does my pipeline work?

Session 3's split, labels and loading, tested end-to-end in minutes, not hours.

3 How much does DL add?

A CNN that beats the baseline by 2 points is a very different story from one that beats it by 20.

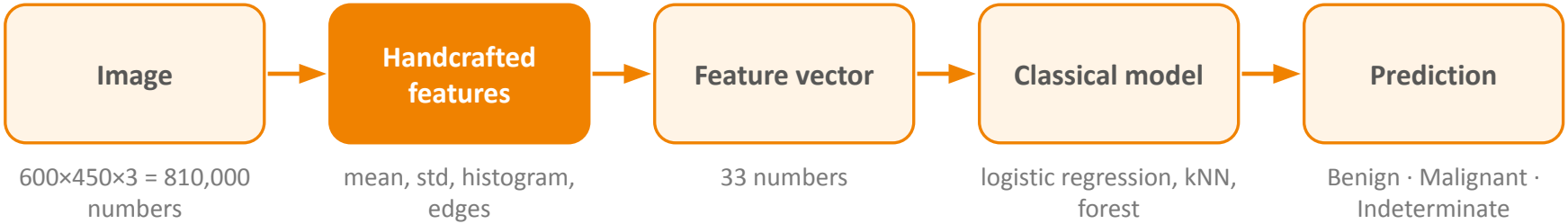
KEY IDEA

Always build the simplest reasonable baseline first.
In Session 7, our CNN's score will be compared to today's number.

SESSION 5 · THEORY

The Classical Vision Pipeline

? *In a CNN, features are learned. Here, who decides what counts as a 'feature'?*



TODAY — CLASSICAL ML

- **We** design the features, using domain knowledge.
- The model only learns the last step:

  features → label.

SESSION 7 — CNN

- The network learns the features **and** the classifier, straight from pixels.
- More powerful, but slower, data-hungry and harder to debug.

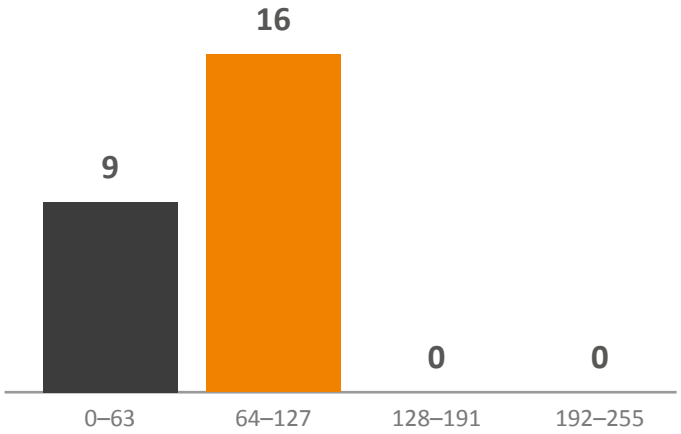
SESSION 5 · THEORY · BY HAND

Color Histogram by Hand

INPUT Session 4's toy, 5×5

100	100	100	100	100
100	100	100	100	100
100	100	10	10	10
100	100	10	10	10
100	100	10	10	10

HISTOGRAM 4 bins, pixel counts



COUNT → FRACTION

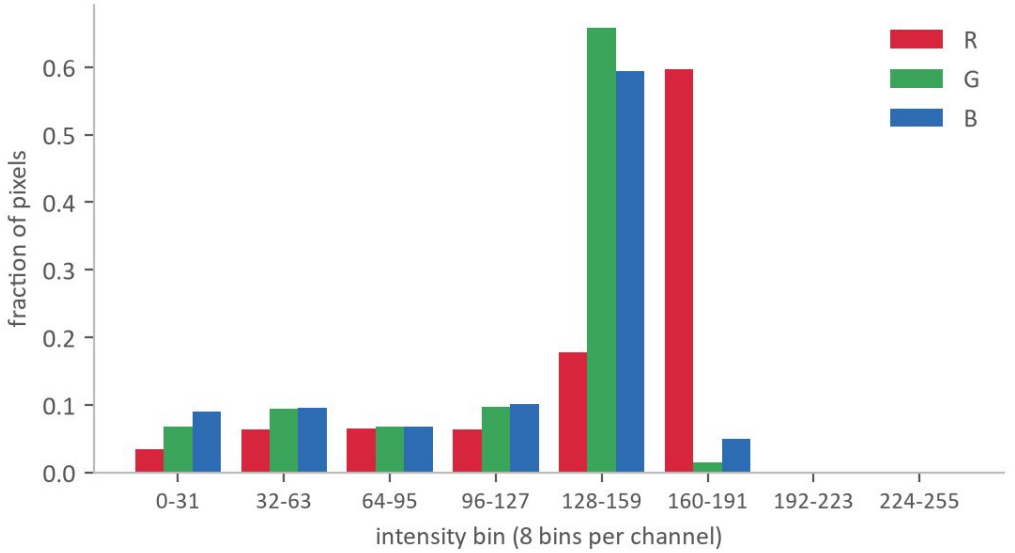
bin	÷ 25 pixels
0–63	9 / 25 = 0.36
64–127	16 / 25 = 0.64
128–191	0
192–255	0

feature = **[0.36, 0.64, 0, 0]**

- Count how many pixels fall into each intensity bin, then **divide by the number of pixels** so big and small images are comparable.
- For color, one histogram per channel: **8 bins × 3 channels = 24 numbers** per image.

Color Histograms on a Real Lesion





- Lesion ISIC_0073863, resized to 200×150. Most pixels sit in the bright bins (skin); the dark tail is the lesion itself.
- Red is shifted to the right of green and blue: skin and lesion are reddish. These **24 bars become 24 features**.

Mean and Standard Deviation by Hand

INPUT 5×5

100	100	100	100	100
100	100	100	100	100
100	100	10	10	10
100	100	10	10	10
100	100	10	10	10

MEAN

$\mu = (16 \cdot 100 + 9 \cdot 10) / 25 = 1690 / 25 =$ **67.6**

STANDARD DEVIATION

$\text{mean}(x^2) = (16 \cdot 10\,000 + 9 \cdot 100) / 25 = 6436$

$\sigma = \sqrt{(\text{mean}(x^2) - \mu^2)} = \sqrt{(6436 - 4569.8)} =$ **43.2**

REAL LESION ISIC_0073863

	R	G	B
mean	142.3	121.9	119.4
std	41.8	42.5	45.5

→ **6 numbers**: 3 means + 3 stds

- **Mean** = overall color: a dark vs. a light lesion.
- **Std** = color variability: one uniform shade vs. many shades.
- Remember **mean(x^2) − mean(x)^2**: we reuse exactly this trick for texture in a few slides.

Same Histogram, Different Image

? *Same 25 numbers, same histogram, same mean and std. Can a color feature tell A from B?*

100	100	100	100	100
100	100	100	100	100
100	100	10	10	10
100	100	10	10	10
100	100	10	10	10

A — toy lesion

10	10	100	10	100
10	100	10	100	100
10	100	100	100	100
100	100	100	10	100
100	100	100	10	10

B — same pixels, shuffled

	A	B
pixels = 10	9	9
pixels = 100	16	16
mean / std	67.6 / 43.2	67.6 / 43.2
changes left→right	3	10

TEXTURE

Histograms and statistics ignore **where** pixels are. **Texture** = how intensity changes from a pixel to its neighbors.
Simplest texture measure: count how many times a pixel differs from its right neighbor
A changes 3 times left→right, *B* changes 10 times.

Texture Feature 1: Edge Density by Hand

Gx Session 4

−90	−90	0
−270	−270	0
−360	−360	0

Gy Session 4

−90	−270	−360
−90	−270	−360
0	0	0

MAGNITUDE $\sqrt{(Gx^2+Gy^2)}$

127.3	284.6	360
284.6	381.8	360
360	360	0

mean of the 9 values
= 2518.3 / 9 = **279.8**
= edge density

- **Edge density** = mean Sobel magnitude over the whole image: one number for 'how many borders and how much fine detail'.
- Warning: hairs are strong edges too. A naive edge feature partly measures **hair**, not lesion.

Three Simple Texture Features

? *Which filter gives which feature?*

Feature	How to compute it	Intuition	From Session 4
Edge density	`np.hypot(sobel_x, sobel_y).mean()`	High = many borders / detail; low = smooth	Sobel Gx, Gy
Roughness	mean of the variance in every 9×9 window	High = rough; low = uniform	box blur
Laplacian variance	`ndimage.laplace(gray).var()`	High = sharp detail; low = blurry	outline kernel

THE TRICK

Local variance = `ndimage.uniform_filter(x²) − ndimage.uniform_filter(x)²`: the same **mean(x²) − mean(x)²** as the std slide, computed in every 9×9 window at once (no Python loop).

A Step Further: Co-occurrence (GLCM) by Hand

A — toy lesion

pixel (rows) × its right neighbor (columns)

	right neighbor 10	right neighbor 100
10	6	0
100	3	11

changes = 0 + 3 = **3** of 20 pairs

B — shuffled

	right neighbor 10	right neighbor 100
10	2	6
100	4	8

changes = 6 + 4 = **10** of 20 pairs

- The **GLCM** counts how often intensity i sits next to intensity j.
- **Diagonal** (orange) = same neighbors, smooth. **Off-diagonal** (blue) = transitions, texture.
- Real images: 8–32 gray levels, 4 directions → contrast, homogeneity, energy, correlation (Haralick, 1973).

OPTIONAL EXTENSION

`skimage.feature.graycomatrix` / `graycoprops` compute this for you.
Today we keep the 3 simple texture features; GLCM is a good idea for your project.

Shape Features: Borrowing from Dermatology (ABCDE)

? *Dermatologists already use a mental checklist to judge lesions. Could you turn that checklist into code?*

ABCDE letter	Clinical meaning	Computable feature
A — Asymmetry	Halves look different	Color difference between left and right halves
B — Border	Irregular, notched border	Edge density; perimeter² / area with a lesion mask
C — Color	Several colors present	Color std, histogram spread, # of color clusters (k-means)
D — Diameter	Larger than ~6 mm	Mask area in pixels (needs a mm-per-pixel scale)

- 'E' (Evolving) needs images over time: not everything clinically relevant is visible in one photo.
- Shape features need a **lesion mask** (segmentation, Session 8). Without one, we use whole-image approximations.

Assembling the Feature Vector

? *Color statistics + histograms + texture: how long is our vector?*

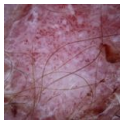


33 numbers per image, **always the same length**, whatever the image size.

That vector is all the model will ever see of the lesion.

From Images to a Table: X and y

? *Look at the numbers: is there ONE column that separates Benign from Malignant?*

image	isic_id	mean_R	std_B	edge_density	roughness	laplacian_var	…	y
	ISIC_0089407	150.3	24.0	34.5	71.4	134.6	…	Benign
	ISIC_0051817	134.0	36.2	67.7	235.5	569.0	…	Malignant
	ISIC_0167803	151.2	76.4	34.0	143.0	124.4	…	Benign
	ISIC_0097302	160.8	19.6	24.1	37.5	101.9	…	Malignant

X: shape (3593, 33), one row per training image · **y**: shape (3593,), the diagnosis. scikit-learn only needs `fit(X, y)` and `predict(X)`.

Feature Scale Matters: Standardize

? *Red ≈ 170, edge density ≈ 29, histogram bins ≈ 0.1. Which one dominates a distance?*

Feature	train mean μ	train std σ	ISIC_0051817 x	z = (x − μ) / σ
mean_R	170.6	23.7	134.0	(134.0 − 170.6) / 23.7 = **−1.54**
edge_density	29.3	13.2	67.7	(67.7 − 29.3) / 13.2 = **2.91**
laplacian_var	176.1	294.7	569.0	(569.0 − 176.1) / 294.7 = **1.33**

- After standardizing, every feature has mean 0 and std 1: 'how unusual is this lesion', in comparable units.
- μ and σ are learned on the **training set only**, then applied to validation and test (Session 3: no leakage).
- Matters for distance-based models, like KNN. Trees split one feature at a time, so they don't care.

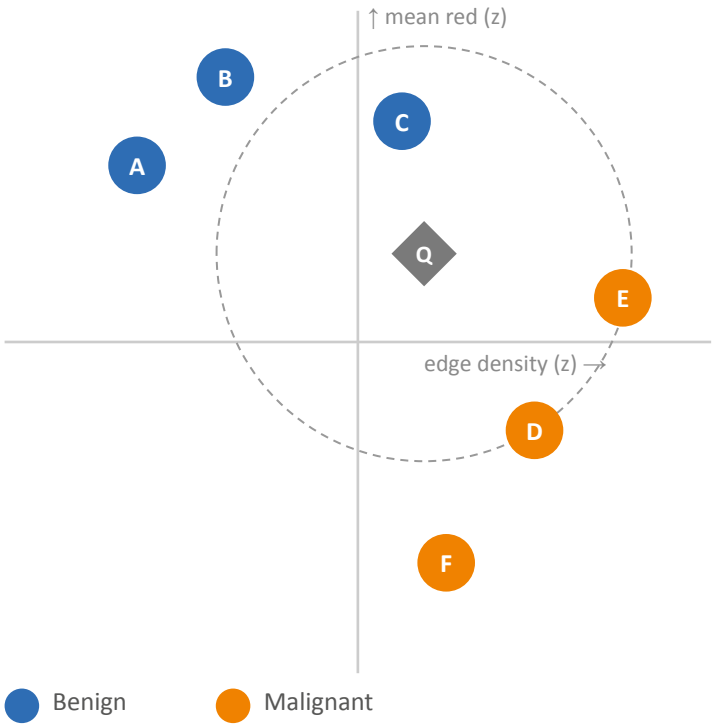
Classical Models: A Quick Tour

? *Same 33 numbers, four different models. Do you expect them all to agree?*

Model	Core idea	Pros / cons
k-Nearest Neighbors	Vote of the k most similar training images	No training; slow to predict; needs scaling
Logistic Regression	Weighted sum of features ⟶ sigmoid ⟶ probability	Fast, interpretable; only linear boundaries
Decision Tree / Random Forest	Yes/no questions on one feature at a time; forest = many trees voting	Non-linear, robust; less interpretable
Support Vector Machine	Boundary with the widest margin; kernels bend it	Strong on small data; slower on large data

Next: three of them by hand, on the same 6 toy lesions and the same new lesion Q.

k-Nearest Neighbors by Hand



point	label	distance to Q	rank
A	Benign	1.36	
B	Benign	1.20	
C	Benign	**0.61**	1
D	Malignant	**0.94**	3
E	Malignant	**0.92**	2
F	Malignant	1.40	

- distance C⟶Q = $\sqrt{(0.1^2 + 0.6^2)} = 0.61$
- **k = 1**: nearest is C ⟶ **Benign**
- **k = 3**: C, E, D ⟶ 1 Benign, 2 Malignant ⟶ **Malignant**

Same lesion, different k, different answer.

Logistic Regression by Hand

MODEL

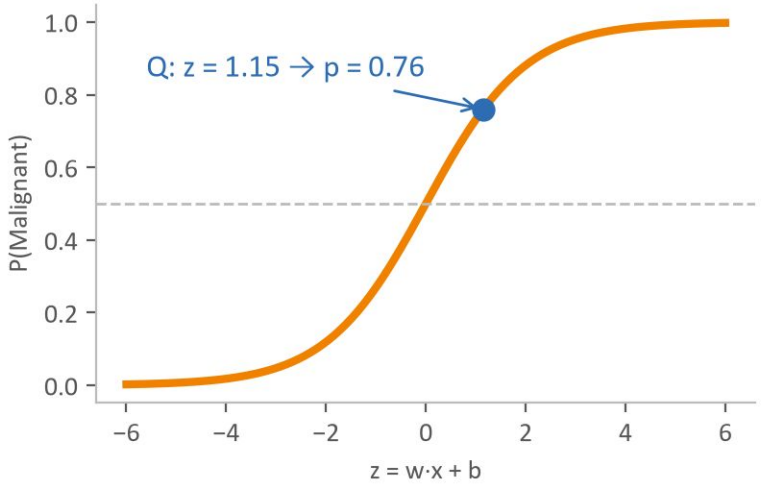
$z = w_1 \cdot \text{edge} + w_2 \cdot \text{red} + b, \quad p = \sigma(z) = 1 / (1 + e^{-z})$

learned: $w_1 = 2.0$, $w_2 = -0.5$, $b = 0.75$

NEW LESION Q = (0.3, 0.4)

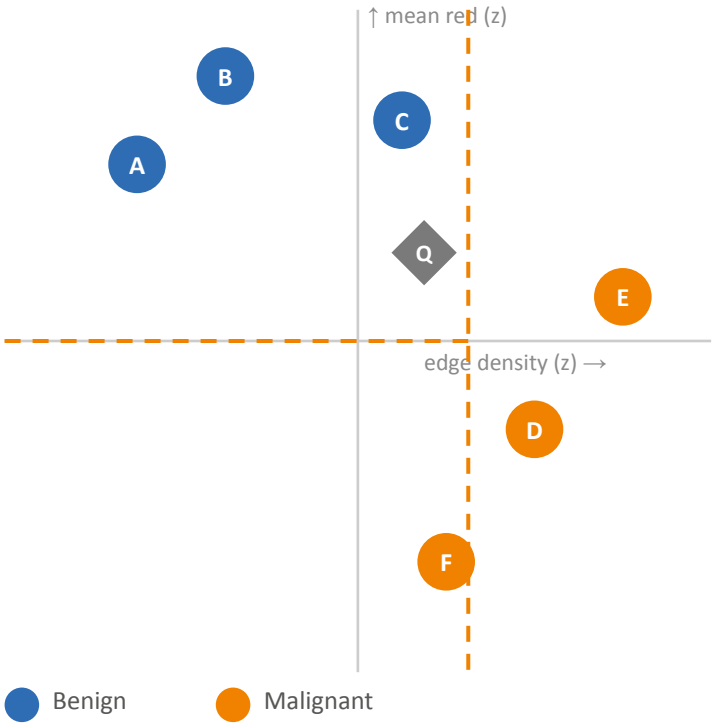
$z = 2.0 \cdot 0.3 - 0.5 \cdot 0.4 + 0.75 =$ **1.15**

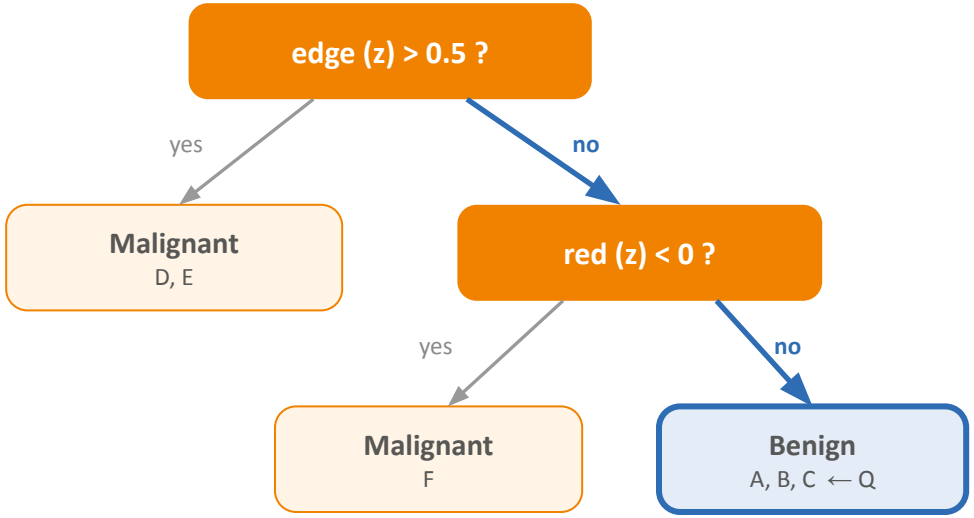
$p = \sigma(1.15) =$ **0.76** ⟶ **Malignant**



- The boundary is where p = 0.5, i.e. z = 0: a **straight line** (a hyperplane in 33-D). Curved patterns are out of reach.
- Weights are readable: $w_1 > 0$ means 'more edges ⟶ more likely Malignant'.
- `class_weight="balanced"`: mistakes on rare classes cost more (remember Session 3's imbalance lesson).

A Decision Tree by Hand





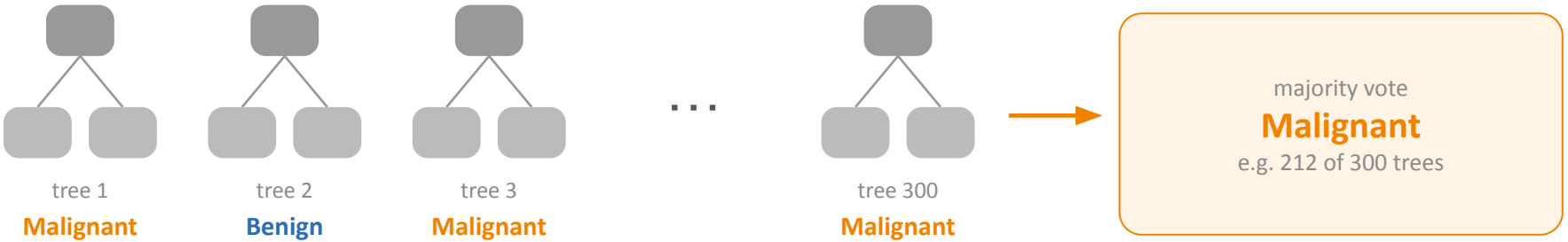
Q = (0.3, 0.4): 0.3 > 0.5? no → 0.4 < 0? no → **Benign**

kNN (k=3): **Malignant** · logistic: **Malignant** (0.76) · tree: **Benign**

Same data, different decision rules, different answers.

Random Forests: Many Trees, One Vote

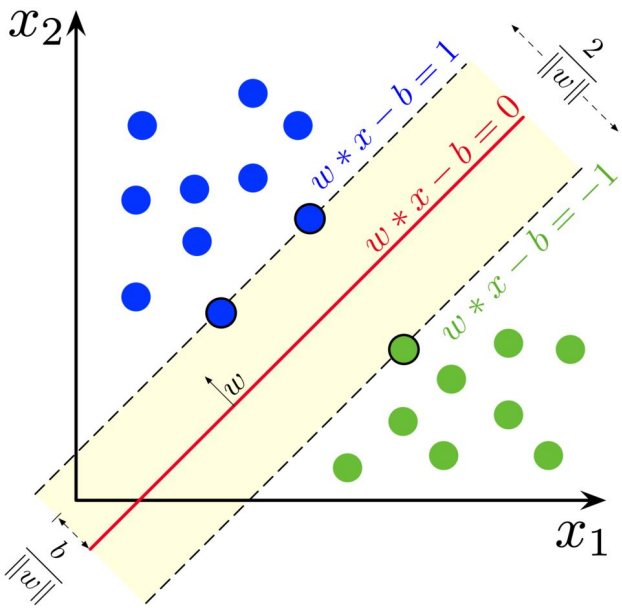
? *One decision tree overfits easily. Does averaging many imperfect trees actually help?*



- Each tree sees a **random sample of images** and a **random subset of features** at every split, so the trees make different mistakes.
- Averaging their votes cancels much of the noise (**variance reduction**): the forest generalizes far better than one tree.
- Strong default for small tabular feature vectors, no scaling needed, and it tells us **which features it used** (feature importances).

Support Vector Machines, Intuition

- An SVM finds the boundary with the **widest margin**: the largest gap to the nearest point of either class.
- Only the closest points, the **support vectors**, define the boundary; the others could move without changing it.
- For curved boundaries, the **kernel trick** (e.g. RBF) implicitly maps features into a higher-dimensional space.
- Often strong on small and medium datasets like ours.



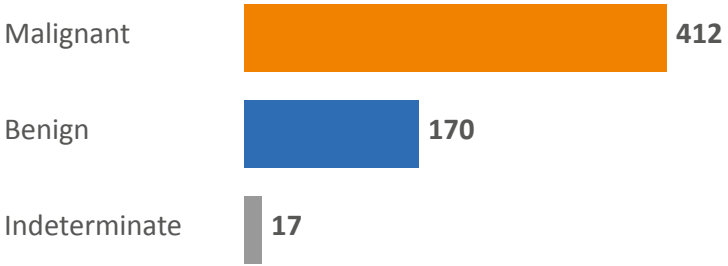
Maximum-margin boundary, support vectors on the dashed lines

SESSION 5 · THEORY · BY HAND

The Dumbest Model: Always Say 'Malignant'

? *A model that never looks at the image. What accuracy does it get on our validation set?*

VALIDATION SET 599 dermoscopic images



68.8%

accuracy: looks decent

0.27

macro-F1: learned nothing

accuracy = 412 / 599 = **0.688**

F1(Malignant) = 2 · 0.688 · 1.0 / (0.688 + 1.0) = 0.815; F1(Benign) = F1(Indeterminate) = 0

macro-F1 = (0.815 + 0 + 0) / 3 = **0.272** → every real model must beat **0.27 macro-F1**, not 68.8% accuracy.

Reading a Confusion Matrix

TRUE \ PREDICTED	Benign	Malignant	Indeterm.
Benign	**41**	6	1
Malignant	9	**78**	3
Indeterm.	2	4	**6**

- Rows = the truth, columns = what the model said. The **diagonal** is correct.
- Accuracy = diagonal / total = (41 + 78 + 6) / 150 = **83.3%**.
- Indeterminate row: only **6 of 12** found. Accuracy hides it, because that class is small.
- Row-wise ⟶ **recall** per class. Column-wise ⟶ **precision** per class.

Precision, Recall and F1 by Hand

PRECISION

TP / column total

Of everything I called X, how much was X?

RECALL

TP / row total

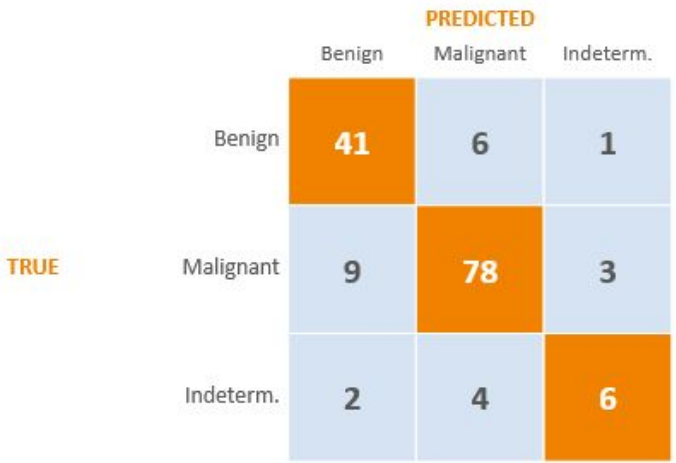
Of all real X, how many did I find?

F1

2·P·R / (P + R)

One number; low if either is low.

class	precision	recall	F1
Benign			
Malignant			
Indeterminate			



Precision, Recall and F1 by Hand

PRECISION

TP / column total

Of everything I called X, how much was X?

RECALL

TP / row total

Of all real X, how many did I find?

F1

2·P·R / (P + R)

One number; low if either is low.

class	precision	recall	F1
Benign	41 / 52 = 0.79	41 / 48 = 0.85	0.82
Malignant	78 / 88 = 0.89	78 / 90 = 0.87	0.88
Indeterminate	6 / 10 = 0.60	6 / 12 = **0.50**	**0.55**

- **macro-F1** = (0.82 + 0.88 + 0.55) / 3 = **0.75**, vs. accuracy 0.83: the weak class pulls it down.
- 'Macro' = plain average over classes: a rare class counts as much as a big one.

The Baseline Recipe



KEY IDEA

A baseline is a **number**, not a model. Its job is to give every later model something honest to beat.

Part 1 Recap: Key Takeaways

1. **Baseline first.** Fast, cheap, and an honest number to compare every later model against.

2. **Color.** Mean, std and histograms summarize color, but ignore where pixels are.

3. **Texture.** Edge density, roughness, Laplacian variance: Session 4's filters, averaged to one number each.

4. **Models.** kNN, logistic regression, trees/forests, SVM: different rules on the same 33 numbers.

5. **Evaluation.** Majority baseline, confusion matrix, macro-F1. Accuracy alone lies on imbalanced data.